1.5 R Markdown and Quarto

Prefer Quarto for new writeups. YAML is the control panel; chunk options decide what the reader sees.

1. Three Kinds of Files

RStudio can edit more than one kind of file. The three you will see in this course are:

File	Extension	What it is for
R script	`.R`	Code you want to keep and rerun
Quarto	`.qmd`	A document that mixes writing and R code. **Prefer this for new writeups.**
R Markdown	`.Rmd`	The older document format with the same idea

A **script** is the right tool when the product is code: a lab, a function, a data-cleaning pipeline.

A **document** is the right tool when the product is something a human should read: a homework writeup, a report, slides. The file holds prose, headings, and code. RStudio can turn that file into HTML, PDF, or Word.

Lab 1 is a script. Later writeups are a good place to use Quarto. Create and render one this week so the habit is already there.

---

2. Why Quarto

Quarto is the current document system from Posit, the same people who make RStudio. It grew out of R Markdown and is the better place to start for new work.

R Markdown still works. You will open `.Rmd` files in older notes, textbooks, and Stack Overflow answers. Knit those files. When you create a new writeup, reach for a `.qmd` file instead.

Quarto is the stronger default because:

- Posit is actively developing it. New features land in Quarto first. R Markdown is stable, but it is no longer where the new work happens.
- One format covers HTML, PDF, Word, and slides with more consistent YAML. You spend less time remembering `html_document` versus `pdf_document`.
- Chunk options sit on `#|` lines, so they are easy to read and change. The older `{r echo=FALSE, warning=FALSE}` fence gets crowded fast.
- Document-wide `execute:` defaults live in the YAML. You set `echo` or `warning` once instead of repeating `knitr::opts_chunk$set()` in a setup chunk.
- The same `.qmd` file can include R now and Python later if a project needs both. R Markdown is built around R.

- HTML output is easier to share. `embed-resources: true` makes a single file you can email or submit, without a folder of extra plots.
- Cross-references, callouts, and diagrams are first-class later in the course. You do not have to switch tools when a writeup gets more ambitious.
- RStudio's **Render** button and the visual editor are built around Quarto.
- Learning Quarto first makes an old `.Rmd` easy to read. The reverse is less true for newer options.

You do not have to convert every file you find. Prefer Quarto when you start something new.

Quarto is a **program** that RStudio calls. It is not an R package. Recent RStudio / Posit Desktop already includes it. If **Render** is missing, install Quarto from quarto.org and reopen RStudio.

You can confirm it is available in RStudio's **Terminal** pane:

```
quarto --version
```

---

3. Create a Quarto Document

1. **File → New File → Quarto Document...**
2. Choose **HTML** as the default format this week.
3. Give it a title and your name.
4. Save it in your project folder with a `.qmd` extension, for example `week01_practice.qmd`.

An `.qmd` file has three parts:

1. a **YAML header** between `---` lines — the control panel for the document
2. **text**, formatted with Markdown
3. **code chunks**, which contain R

A small file looks like this:

````
---
title: "Week 1 practice"
author: "Your Name"
format: html
---

# A heading

This is ordinary writing. R can compute inline with an `r` expression inside backticks.

```{r}
x <- 1:5
mean(x)
```
````

Notice `format: html`, not `output: html_document`. That is the Quarto header. `format: pdf` and `format: docx` are the usual alternatives.

Insert a code chunk with **Ctrl+Alt+I** (Windows/Linux) or **Cmd+Option+I** (macOS), or **Code → Insert Chunk**.

Run a line or a chunk with **Ctrl+Enter** / **Cmd+Enter**, the same way you run a script. Results appear in the console, and often under the chunk.

---

4. Render

When you want a document, click **Render** (not **Knit**). Rendering:

- saves the file
- reads the YAML header
- runs the code in a **fresh** R session, not from leftover console objects
- writes an HTML (or PDF, or Word) file next to the `.qmd`

That fresh session is the point. Everything the document needs must be in the file: `library()` calls, data import, and the code that produces the numbers you quote.

Do **not** put `install.packages()` in the document. Install packages once in the console, as in note **1.2**.

If rendering to PDF fails, you are missing a LaTeX installation. Render to HTML this week. PDF can wait until a later assignment actually requires it. When that happens, a common path is:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

then close and reopen RStudio.

---

5. Markdown You Will Use Constantly

The same Markdown works in Quarto and in R Markdown:

```
# Heading
## Subheading

*italic*  **bold**  `code`

- a list
- of items

1. a numbered list
1. the numbers update themselves

[a link](https://example.com)
```

A single Enter does not start a new paragraph. Leave a blank line, or end a line with two spaces.

YAML is picky about spacing. If Render fails with a YAML error, look at indentation around `format:`.

---

6. What YAML Does

YAML is the block at the top of the file, between two lines of `---`.

It does **not** run R. It tells Quarto (or R Markdown) how to turn the file into a document:

- who wrote it and what it is called
- which output to build (HTML, PDF, Word, slides)
- how that output should look (table of contents, numbered headings, page numbers)
- default rules for every code chunk

Think of it as the cover sheet and the settings panel. The Markdown and the chunks are the content. YAML is the instructions for assembling that content.

A key is a word to the left of a colon. A value is to the right:

```
title: "Week 1 practice"
```

Some keys have nested options. Those child lines must be indented. Two extra spaces, or a missing space after a colon, will stop Render.

Quarto uses hyphens in option names (`number-sections`). R Markdown often uses underscores (`number_sections`). The idea is the same; the spelling is not.

---

7. Common YAML Options

These are the options you will actually use this semester.

Identity

```
---
title: "Lab 2"
author: "Your Name"
date: today
format: html
---
```

`date: today` inserts the date when you Render. You can also write a fixed date in quotes.

The output format

```
format: html
```

or, when you need options under that format:

```
format:
  html:
    toc: true
```

`format`	What you get
`html`	A web page. Best default this week.
`pdf`	A paginated PDF. Needs LaTeX.
`docx`	A Word file.
`revealjs`	HTML slides.

Table of contents and numbered headings

```
format:
  html:
    toc: true
    toc-depth: 2
    number-sections: true
```

- `toc: true` adds a table of contents from your `#` and `##` headings.
- `toc-depth: 2` includes headings down to `##`, not `###`.
- `number-sections: true` numbers the headings: 1, 1.1, 2, 2.1.

`number-sections` numbers **headings**, not pages.

Page numbers

HTML is one scrolling page. It does not have page numbers.

A **PDF** is paginated. Page numbers appear automatically in the footer once LaTeX is installed and you render with `format: pdf`. You do not add them by hand.

```
format:
  pdf:
    toc: true
    number-sections: true
    papersize: letter
```

`papersize: letter` is the usual US choice. `number-sections` still means heading numbers. The printed page numbers come with the PDF itself.

For HTML slides (`format: revealjs`), slide numbers are a separate option:

```
format:
  revealjs:
    slide-number: true
```

We will not need slides in week 1.

One HTML file you can email

```
format:
  html:
    embed-resources: true
```

That folds CSS and plots into a single `.html` file, which is easier to submit or send than a folder of extra files.

Default rules for every chunk

YAML can set chunk options for the whole document. Later you override one chunk if you need to.

```
---
title: "Lab 2"
author: "Your Name"
format: html
execute:
  echo: true
  warning: false
  message: false
---
```

`execute:` is a Quarto key. It is not the same as `eval` on one chunk. It means: "unless a chunk says otherwise, do this."

A compact header students often start from:

```
---
title: "Week 1 practice"
author: "Your Name"
date: today
format:
  html:
    toc: true
    number-sections: true
    embed-resources: true
execute:
  echo: true
  warning: false
---
```

---

8. Code Chunks: `echo`, `eval`, and `include`

A code chunk is a fenced block of R. Quarto options go on `#|` lines at the top of the chunk:

````
```{r}
#| echo: true
#| eval: true
#| include: true

mean(1:5)
```
````

Three options decide what happens. Learn these first.

Option	Question it answers	Default
`echo`	Does the reader see the code?	`true`
`eval`	Does R run the code?	`true`
`include`	Does anything from this chunk appear in the document?	`true`

They combine. That is the point.

`echo`	`eval`	`include`	The reader sees	R runs the code
true	true	true	code and results	yes
false	true	true	results only	yes
true	false	true	code only	no
true	true	false	nothing	yes
false	true	false	nothing	yes

When to use each combination:

- **Show code and results** (`echo: true`). The default. Good for homework where we want to see how you got the number.
- **Results only** (`echo: false`). A report for a reader who does not need the code.
- **Code only** (`eval: false`). An example you want people to read but not run: it would take too long, or it is incomplete on purpose.
- **Run, but hide everything** (`include: false`). A setup chunk: `library(tidyverse)`, reading `students.csv`. The later chunks need those objects. The reader does not need that boilerplate.

`include: false` still **evaluates** the chunk unless you also set `eval: false`. The objects remain available.

A usual start-of-file setup chunk:

````
```{r}
#| label: setup
#| include: false

library(tidyverse)
students <- read_csv("data/students.csv")
```
````

Inline R still uses an `r` expression inside backticks. Use that for a number you have already computed, so the writeup updates when you Render again.

---

9. Other Chunk Options You Will Meet

After `echo`, `eval`, and `include`, these are the next ones worth knowing.

Option	What it does
`output`	If `false`, hide printed results but still run the chunk. Different from `include: false`, which also hides the code.
`warning`	If `false`, hide warnings. Useful for package startup noise.
`message`	If `false`, hide messages such as `read_csv()` column types.
`error`	If `true`, Render continues even if the chunk errors, and the error is printed. Leave this off unless you were asked to show a failing line.
`label`	A name for the chunk, such as `setup` or `mean-grade`. Labels must be unique.
`fig-width`, `fig-height`	Figure size, in inches.
`fig-cap`	A caption under the figure.

Example:

````
```{r}
#| label: grade-mean
#| echo: false
#| warning: false
#| fig-width: 6
#| fig-height: 4
#| fig-cap: "Grades in the practice file."

mean(students$grade)
```
````

You do not need all of these in week 1. Put `library(tidyverse)` in an early chunk so every later chunk can use it. Use `echo` to control whether we see your code. Use `include: false` for setup.

A chunk option overrides the document `execute:` default for that one chunk only.

---

10. The Same Options in R Markdown

R Markdown uses the same three ideas. The spelling and the button are different.

The header uses `output:` instead of `format:`, and underscores instead of hyphens:

```
---
title: "Week 1 practice"
author: "Your Name"
output:
  html_document:
    toc: true
    number_sections: true
  pdf_document:
    toc: true
    number_sections: true
---
```

Chunk options often sit in the fence line instead of on `#|` lines:

````
```{r echo=FALSE, warning=FALSE, include=TRUE}
mean(1:5)
```
````

Document-wide defaults look like this, usually in a setup chunk:

```
knitr::opts_chunk$set(
  echo = TRUE,
  warning = FALSE,
  message = FALSE
)
```

The button is **Knit**, not **Render**. The idea is the same: Markdown plus code chunks, run in a fresh session.

If an old assignment or a book gives you a `.Rmd` file, knit it. There is no need to rewrite it as Quarto. When you start a new writeup, prefer a `.qmd` file. Quarto will also accept the older `{r echo=FALSE}` fence style if you open an old example.

Jupyter notebooks are a third interactive format, used heavily in Python. We will not use them here.

---

11. A Sample Report

A complete `.qmd` is easier to learn from than isolated snippets. Download sample-quarto-report.qmd and Render it in an RStudio project.

The report uses the built-in `mtcars` data, so you do not need `students.csv`. Open the source and read the comments. They mark:

- the YAML header (`toc`, `number-sections`, `execute`)
- a setup chunk with `include: false`
- inline R for numbers that should not be typed by hand

- cross-references (`@tbl-vars`, `@fig-hist`, `@sec-extensions`)
- two plots side by side (`layout-ncol`)
- a short LaTeX equation

You also need `broom` for the model table. Install it once in the console if R says the package is missing:

```
install.packages("broom")
```

If Render to PDF fails, change `format: pdf` to `format: html` and Render again. A rendered PDF of the sample is posted so you can see the intended result.

Lab 1 does not need this file.

---

12. What to Use When

```
Need to keep code and rerun it?
  → .R script

Need a readable writeup with results baked in?
  → start a .qmd (Quarto)

Someone handed you an old .Rmd?
  → knit it; no need to rewrite it

Lab 1
  → lab01.R

Later homework
  → Quarto is the better default; an assigned .Rmd is still fine
```

Three controls to remember:

```
YAML              → how the document is built
echo / eval / include → what each chunk shows and whether it runs
number-sections   → heading numbers
PDF format        → printed page numbers
```

Note **1.1** is how you run a script and find the working directory. Note **1.3** is how you import `students.csv`. Lab 1 uses both. Lab 1 does not need this note.